

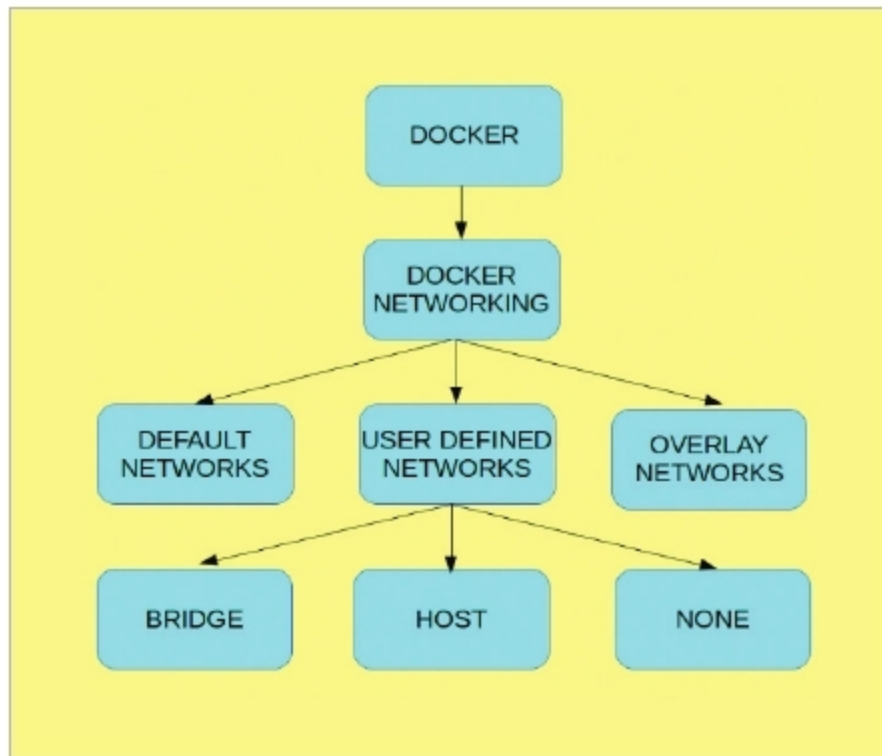


Figure 1: Block diagram of Docker networking

```

➔ ~ service docker status
● docker.service - Docker Application Container Engine
   Loaded: loaded (/lib/systemd/system/docker.service; enabled;
   Active: active (running) since Thu 2018-04-19 17:51:49 IST; 1
   Docs: https://docs.docker.com
  Main PID: 1627 (dockerd)
    Tasks: 34
   Memory: 131.6M
     CPU: 27.793s
    CGroup: /system.slice/docker.service
            └─1627 /usr/bin/dockerd -H fd://
               2365 docker-containerd -l unix:///var/run/docker/li

Apr 19 17:51:47 shreyakupadhyay-Inspiron-5547 dockerd[1627]: tin
Apr 19 17:51:48 shreyakupadhyay-Inspiron-5547 dockerd[1627]: tin
Apr 19 17:51:48 shreyakupadhyay-Inspiron-5547 dockerd[1627]: tin
Apr 19 17:51:49 shreyakupadhyay-Inspiron-5547 dockerd[1627]: tin
Apr 19 17:51:49 shreyakupadhyay-Inspiron-5547 dockerd[1627]: tin
Apr 19 17:51:49 shreyakupadhyay-Inspiron-5547 dockerd[1627]: tin

```

Figure 2: To check if Docker is running

to Docker containers, (ii) the creation of Docker containers in a specified IP address range, and (iii) establishing communication amongst containers that are running on different Docker hosts (multi-host Docker networking). Figure 1 depicts the topology of Docker networking.

Why Docker and not virtual machines?

We will not use virtual machines to perform the above tasks because there are already many open source and commercial tools such as VirtualBox and VMware, which can be used to set up network configuration on virtual machines. They also provide good user interfaces. So, performing the above tasks using a VM is not a big challenge, but with Docker, the community is still growing. As per our experience, no such tools are available directly. Docker only provides command line support for dealing with network configurations.

We assume that Docker is installed and running on your machine. Just like any other service in Linux, we can check if

```

➔ ~ docker network ls
NETWORK ID          NAME                DRIVER
b485643037bd        bridge              bridge
fe56b3e2d73d        docker_gwbridge     bridge
5d989ebbd9c9        host                host
98a251d3ef9f        none                null

```

Figure 3: Listing the default Docker networks

it is running by using the `$ service docker status` command. The output should be similar to what is shown in Figure 2. We can also see other information like the process ID of the Docker daemon, the memory, etc.

If Docker is not already installed, the links [f](#) will guide you to understand the concepts and its installation steps.

Docker networking

Docker creates various default networks, which are similar to what we get in VMs, by using tools like VirtualBox, VMware and Kernel Virtual Machine (KVM). To get a list of all the default networks that Docker creates, run the command as shown in Figure 3.

As can be seen in this figure, there are three types of networks:

1. Bridged network
2. Host network
3. None network

We can assign any one of the networks to the Docker containers. The `--network` option of the 'docker run' command is used to assign a specific network to the container:

```
$ docker run --network=<network name>
```

In Figure 4, the network type of the container is given as 'host' by using the `--network` option of the 'docker run' command. The network assigned to the container is *demo1* using the `--name` option and the docker image is *ubuntu*. '-i' is for interactive, '-t' is for pseudo-TTY, and '-d' is the detach option. This runs the container in the background and prints the new container ID (see Figure 3).

The following command gives detailed information about a particular network.

```
$ docker network inspect <network name>
```

From Figure 5, we have configured the network to be the host. So, we will get the information about the host network and the container details that we created earlier. We can see that it gives the output in JSON format with information about the containers connected to that network, the subnet and gateway of that network, the IP address and MAC address of each of the containers connected to that network, and much more. The container *demo1*, which we created above, is shown in Figure 5.